

Unicode in the Library, Part 1: UTF Transcoding

Document #: P2728R7
Date: 2024-10-06
Project: Programming Language C++
Audience: SG-16 Unicode
SG-9 Ranges
LEWG
Reply-to: Eddie Nolan
<eddiejnolan@gmail.com>

Contents

- 1 Changelog
 - 1.1 Changes since R0
 - 1.2 Changes since R1
 - 1.3 Changes since R2
 - 1.4 Changes since R3
 - 1.5 Changes since R4
 - 1.6 Changes since R5
 - 1.7 Changes since R6
- 2 Motivation
- 3 The shortest Unicode primer imaginable
- 4 Basic examples
 - 4.1 Transcoding a UTF-8 string literal to a `std::u32string`
 - 4.2 Sanitizing potentially invalid Unicode
 - 4.3 Returning the final non-ASCII code point in a string, transcoding backwards lazily:
 - 4.4 Transcoding strings and throwing a descriptive exception on invalid UTF
 - 4.5 Adapting a range of non-character-type values
- 5 Proposed design
 - 5.1 Dependencies
 - 5.2 Discussion of whether transcoding views should accept ranges of `char` and `wchar_t`
 - 5.3 Error handling mechanism
 - 5.3.1 Why `std::expected<void, E>?`
 - 5.3.2 Existing practice

- 5.3.3 `std::uc::transcoding_error` enumerators
- 5.3.4 Examples
- 5.4 Erroneous Behavior
- 5.5 Optimization for transcoding views wrapping other transcoding views
- 5.6 Other design notes
- 5.7 Null-terminated sequence sentinel `null_sentinel` and associated CPO `null_term`
- 5.8 Exposition-only concepts and traits
- 5.9 Transcoding views
- 5.10 Add code unit views and adaptors
- 5.11 Why there are three `to_utfN_views` views plus `utf_view`, and three `as_charN_t_views`
- 5.12 Why `as_charN_t_view` is not implemented in terms of `transform_view`
- 5.13 Why `utf_view` always transcodes, even in UTF-N to UTF-N cases
- 5.14 Add a feature test macro
- 5.15 Relevant Polls/Minutes
 - 5.15.1 SG16 review of P2728R7 on 2023-09-13 (Telecon)
 - 5.15.2 SG16 review of P2728R6 on 2023-08-23 (Telecon)
 - 5.15.3 SG9 review of D2728R4 on 2023-06-12 during Varna 2023
 - 5.15.4 SG16 review of P2728R3 on 2023-05-10 (Telecon)
 - 5.15.5 SG16 review of P2728R0 on 2023-04-12 (Telecon)
 - 5.15.6 SG16 review of P2728R0 on 2023-03-22 (Telecon)
- 6 Implementation experience
- 7 Appendix: Implementing Existing Practice for Error Handling
 - 7.1 `iconv`
 - 7.2 ICU `u_strFromUTF8WithSub`
 - 7.3 Windows `MultiByteToWideChar`
 - 7.4 Python `decode()`
- 8 Special Thanks
- 9 References

§ 1 Changelog

§ 1.1 Changes since R0

- When naming code points in interfaces, use `char32_t`.
- When naming code units in interfaces, use `charN_t`.
- Remove each eager algorithm, leaving in its corresponding view.
- Remove all the output iterators.

- Change template parameters to `utfN_view` to the types of the from-range, instead of the types of the transcoding iterators used to implement the view.
- Remove all make-functions.
- Replace the misbegotten `as_utfN()` functions with the `as_utfN` view adaptors that should have been there all along.
- Add missing `transcoding_error_handler` concept.
- Turn `unpack_iterator_and_sentinel` into a CPO.
- Lower the UTF iterator concepts from bidirectional to input.

§ 1.2 Changes since R1

- Reintroduce the transcoding-from-a-buffer example.
- Generalize `null_sentinel_t` to a non-Unicode-specific facility.
- In utility functions that search for ill-formed encoding, take a range argument instead of a pair of iterator arguments.
- Replace `utf{8,16,32}_view` with a single `utf_view`.

§ 1.3 Changes since R2

- Add `noexcept` where appropriate.
- Remove non-essential constants and utility functions, and elaborate on the usage of the ones that remain.
- Note differences from similar elements proposed in [P1629R1].
- Extend the examples slightly.
- Correct an error in the description of the view adaptors' semantics, and provide several examples of their use.

§ 1.4 Changes since R3

- Changed the definition of the `code_unit` concept, and added `as_charN_t` adaptors.
- Removed the utility functions and Unicode-related constants, except `replacement_character`.
- Changed the constraint on `utf_iterator` slightly.
- Change `null_sentinel_t` back to being Unicode-specific.

§ 1.5 Changes since R4

- Replace `unpacking_owning_view` with `unpacking_view`, and use it to do unpacking, rather than sometimes doing the unpacking in the adaptor.
- Ensure `const` and non-`const` overloads for `begin` and `end` in all views.
- Move `null_sentinel_t` to `std`, remove its base member function, and make it useful for more than just pointers, based on SG-9 guidance.

§ 1.6 Changes since R5

- Simplify the complicated constraint on the comparison operator for `null_sentinel_t`.
- Introduce `ranges::project_view`, and implement `charN_views` in terms of that.
- Convert the `utfN_views` to aliases, rather than individual classes.

§ 1.7 Changes since R6

- Fix a bug in `null_sentinel_t` causing it not to satisfy `sentinel_for` by changing its `operator==` to return `bool`.
- Fix a bug in `null_sentinel_t` where it did not support non-copyable input iterators by having `operator==` take input iterators by reference.
- Rename `as_utfN` to `to_utfN` to emphasize that a conversion is taking place and to contrast with the code unit views, which remain named `as_charN_t`.
- Refactor `utf_view` into an exposition-only `utf-view-impl` class used as an implementation detail of separate `to_utf8_view`, `to_utf16_view`, and `to_utf32_view` classes, addressing broken deduction guides in the previous revision.
- Remove `project_view` and copy most of its implementation into separate `char8_view`, `char16_view`, and `char32_view` classes, addressing broken deduction guides in the previous revision.
- Change `utf_iterator` to an exposition-only member class of `utf-view-impl`.
- Eliminate iterator unpacking mechanism and replace it with an alternative solution to the problem of transcoding ranges wrapping other transcoding ranges. This simplifies the API at the expense of removing the transcoding iterator's `begin()` and `end()` member functions and losing the ability to implement unpacking for user-defined UTF iterators.
- Remove `std::uc::format`.
- Make all concepts exposition-only.
- Remove `transcoding_error_handler` mechanism.
- Introduce new error handling mechanism based on a new `transcoding_error` enumeration which is returned by an `success()` member function of the transcoding view's iterator.
- Remove ability to pass pointers to range adaptor closure objects, which violated the restriction that only ranges may be passed to range adaptor closure objects.
- Remove `std::format` and `std::ostream` functionality. It doesn't make sense for this mechanism to be the only way we have to format/output `char8_t`; we can revisit this functionality when we have already figured out how to support e.g. `std::u8string`.
- Replace code examples with new ones reflecting API changes.
- Provide a reference implementation.

§ 2 Motivation

Unicode is important to many, many users in everyday software. It is not exotic or weird. Well, it's weird, but it's not weird to see it used. C and C++ are the only major production languages with essentially no support for Unicode.

Let's fix.

To fix, first we start with the most basic representations of strings in Unicode: UTF. You might get a UTF string from anywhere; on Windows you often get them from the OS, in UTF-16. In web-adjacent applications, strings are most commonly in UTF-8. In ASCII-only applications, everything is in UTF-8, by its definition as a superset of ASCII.

Often, an application needs to switch between UTFs: 8 -> 16, 32 -> 16, etc. In SG-16 we've taken to calling such UTF-N -> UTF-M operations “transcoding”.

This paper provides interfaces to do UTF transcoding based on the ranges API.

A particular reason for urgency in adding transcoding operations to the standard library is that the standard library has previously contained problematic-to-broken UTF transcoding facilities in the form of `std::codecvt` facets which are currently slated for removal without replacement as [P2871R3] and [P2873R2] make their way through the committee. GitHub searches show that these facilities are widely used; the functionality contained in this paper can serve as a proper replacement.

§ 3 The shortest Unicode primer imaginable

There are multiple encoding types defined in Unicode: UTF-8, UTF-16, and UTF-32.

A *code unit* is the lowest-level datum-type in your Unicode data. Examples are a `char8_t` in UTF-8 and a `char32_t` in UTF-32.

A *code point* is a 32-bit integral value that represents a single Unicode value. Examples are U+0041 “A” “LATIN CAPITAL LETTER A” and U+0308 “” “COMBINING DIAERESIS”.

A code point may be consist of multiple code units. For instance, 3 UTF-8 code units in sequence may encode a particular code point.

§ 4 Basic examples

§ 4.1 Transcoding a UTF-8 string literal to a `std::u32string`

```
std::u32string hello_world =  
    u8"こんにちは世界" | std::uc::to_utf32 | std::ranges::to<std::u32string>();
```

§ 4.2 Sanitizing potentially invalid Unicode

Here, we sanitize potentially invalid Unicode C strings by replacing invalid code units with replacement characters according to Unicode’s recommended Substitution of Maximal Subparts:

```
template <typename CharT>  
std::basic_string<CharT> sanitize(CharT const* str) {  
    return std::uc::null_term(str) | std::uc::to_utf<CharT> |  
        std::ranges::to<std::basic_string<CharT>>();  
}
```

§ 4.3 Returning the final non-ASCII code point in a string, transcoding backwards lazily:

```
std::optional<char32_t> last_nonascii(std::ranges::view auto str) {  
    for (auto c : str | std::uc::to_utf32 | std::views::reverse  
        | std::views::filter([](char32_t c) { return c > 0x7f; }))
```

```

        | std::views::take(1)) {
    return c;
}
return std::nullopt;
}

```

§ 4.4 Transcoding strings and throwing a descriptive exception on invalid UTF

(This example assumes the existence of the `enum_to_string` function from [P2996R5])

```

template <typename FromChar, typename ToChar>
std::basic_string<ToChar> transcode_or_throw(std::basic_string_view<FromChar> input)
{
    std::basic_string<ToChar> result;
    auto view = input | to_utf<ToChar>;
    for (auto it = view.begin(), end = view.end(); it != end; ++it) {
        if (it.success()) {
            result.push_back(*it);
        } else {
            throw std::runtime_error("error at position " +
                                     std::to_string(it.base() - input.begin()) + ": " +
                                     enum_to_string(it.success().error()));
        }
    }
    return result;
}

```

§ 4.5 Adapting a range of non-character-type values

Let’s say that we want to take code points that we got from ICU, and transcode them to UTF-8. The problem is that ICU’s code point type is `int`. Since `int` is not a character type, it’s not deduced by `to_utf8` to be UTF-32 data. We can address this by using the `std::uc::as_char32_t` to cast the `ints` to `char32_t`:

```

std::vector<int> input = get_icu_code_points();
// This is ill-formed without the as_char32_t adaptation.
auto input_utf8 =
    input | std::uc::as_char32_t | std::uc::to_utf8 | std::ranges::to<std::u8string>();

```

§ 5 Proposed design

§ 5.1 Dependencies

This proposal depends on the existence of [P2727R4] “`std::iterator_interface`”.

§ 5.2 Discussion of whether transcoding views should accept ranges of `char` and `wchar_t`

Here are some examples of the differences between having the transcoding views accept ranges of `char` and `wchar_t` or reject them. The `to_utfN` and `as_charN` adaptors are discussed later in this paper.

The `to_utfN` adaptors produce `to_utfN_views`, which do transcoding.

The `as_charN_t` adaptors produce `as_charN_views` that are each very similar to a `transform_view` that casts each element of the adapted range to a `charN_t` value. An `as_charN_view` differs from the

equivalent transform in that it may be a borrowed range.

Note the use of the shorthand “charN_t” below with `std::wstring`. That’s there because whether you write `as_char16_t` or `as_char32_t` is implementation-dependent.

Rejecting ranges of <code>char</code> and <code>wchar_t</code>	Accepting ranges of <code>char</code> and <code>wchar_t</code>
<pre>using namespace std::uc; auto v1 = u8"text" to_utf32; // Ok. auto v2 = u"text" to_utf8; // Ok. auto v3 = U"text" to_utf16; // Ok. auto v4 = std::u8string(u8"text") to_utf32; // Ok. auto v5 = std::u16string(u"text") to_utf8; // Ok. auto v6 = std::u32string(U"text") to_utf16; // Ok. auto v7 = std::string to_utf32; // Error; ill-formed. auto v8 = std::wstring to_utf8; // Error; ill-formed. auto v9 = std::string as_char8_t to_utf32; // Ok. auto v10 = std::wstring as_charN_t to_utf8; // Ok.</pre>	<pre>using namespace std::uc; auto v1 = u8"text" to_utf32; // Ok. auto v2 = u"text" to_utf8; // Ok. auto v3 = U"text" to_utf16; // Ok. auto v4 = std::u8string(u8"text") to_utf32; // Ok. auto v5 = std::u16string(u"text") to_utf8; // Ok. auto v6 = std::u32string(U"text") to_utf16; // Ok. auto v7 = std::string to_utf32; // Ok. auto v8 = std::wstring to_utf8; // Ok. auto v9 = std::string as_char8_t to_utf32; // Ok. auto v10 = std::wstring as_charN_t to_utf8; // Ok.</pre>

In short, rejecting `char` and `wchar_t` forces you to write “`| as_char8_t`” everywhere you want to use a `std::string` with the interfaces proposed in this paper.

SG-16 has previously expressed strong support for rejecting `char` and `wchar_t`, as can be observed in the polling history section.

The feeling in SG-16 was that the `charN_t` types are designed to represent UTF encodings, and `char` is not. A `char const *` string could be in any one of dozens (hundreds?) of encodings. The addition of “`| as_char8_t`” to adapt ranges of `char` is meant to act as a lexical indicator of user intent.

The authors believe this decision is a mistake. Our argument for accepting ranges of `char` and `wchar_t` is as follows.

First, note that none of the `charN_t` types imposes any invariant that a range of its contents contains valid Unicode. As a result, they cannot enforce preconditions for APIs that require valid Unicode input at the level of the type system.

Therefore, we claim that the main use case of the `charN_t` types in APIs is to facilitate a coding style that allows APIs to advertise to users whether they expect Unicode-encoded strings (whether with a wide or a narrow contract).

For example, users of this coding style may write an API like the following:

```
// Expect input to be in Windows-1252
std::size_t word_count(std::string_view);
```

```
// Expect input to be in Unicode
std::size_t word_count(std::u8string_view);
```

If `to_utfN` rejects ranges of `char` and `wchar_t`, it would bring this standard library API into alignment with this style.

However, there are a number of reasons why we consider this approach undesirable for our use case.

First of all, for any large C++ API surface dealing with Unicode that was not designed very recently, there will be APIs that expect UTF-8 in the form of `std::string` parameters. This means that the semantic value of `char8_t` is one-sided; in such an ecosystem, while the presence of `char8_t` certainly indicates that the API expects UTF-8, the absence of `char8_t` may still indicate a `char`-based API that also expects UTF-8.

Furthermore, because `char8_t` is such a recent addition to the standard, and because it's so poorly supported by other standard library facilities such as `<iostream>` and `std::format`, its penetration has been extremely low; a Github Code Search showed 15.3M references to `std::string` and 6.7k references to `std::u8string`.

Finally, due to the particular history of implementation choices by compiler writers, the proportion of C++ users who have the ability to properly benefit from the use of `char8_t` is unfortunately smaller than intended.

For the vast majority of users of Unix-like operating systems, both the basic literal encoding and the execution encoding are UTF-8, and so `char8_t` is mostly redundant, since it has approximately the same meaning as `char`. This leaves Windows developers as the remaining large pool of users who could potentially take advantage of `char8_t`.

The issue is that Windows users are divided into two categories: those who use MSVC's `/utf8` compiler flag, and those who do not.

Users of `/utf8` are in the future: `/utf8` switches the basic literal encoding and execution encoding to UTF-8. These users have less need for `char8_t` because their `chars` are UTF-8.

Non-users of `/utf8` are dealing with non-Unicode basic literal and execution encodings, so theoretically they're the target audience for `char8_t`. But unfortunately, without the `/utf8` flag, MSVC breaks compliance with the standard, in that it violates the requirement that `u8""` string literals are encoded in Unicode. Attempting to create such a string literal on MSVC without specifying `/utf8` results in Windows-1252 code units inside of `char8_t` bytes. For these users, `char8_t` is theoretically useful but broken in practice.

Rejecting `char` and `wchar_t` for UTF transcoding will therefore have limited benefits. On the other hand, rejecting these types will send users over to Stack Overflow to discover they need to copy boilerplate called `| std::uc::as_char8_t` for reasons that will seem academic to most of them.

§ 5.3 Error handling mechanism

When invalid code units are encountered, the UTF transcoding views replace those code units with U+FFFD replacement characters according to the Unicode standard's recommended "Substitution of Maximal Subparts" algorithm.

However, users of the transcoding views may want to know when invalid code units have been encountered, and to implement custom behaviors if this is the case. Simply checking whether the

transcoded code points contain U+FFFD replacement characters is not sufficient because these characters are an in-band signal that can also appear in valid UTF.

What's called for is a basis operation with which arbitrary error handling approaches may be implemented.

The UTF transcoding views in this paper provide such a basis operation by adding an `success()` member function to the iterator of the transcoding view, which informs users whether the current code point is a U+FFFD that was inserted in response to an invalid code unit sequence. The `success()` member function returns a `std::expected<void, std::uc::transcoding_error>`, where `std::uc::transcoding_error` is a new enum class containing enumerators for every category of transcoding error.

Users who choose not to implement error handling will simply sanitize any invalid code unit sequences using U+FFFD replacement. Users who want to implement error handling can implement any of the following approaches, either by wrapping the iterator or by iterating with a traditional for loop:

- Throwing an exception
- Using an character other than U+FFFD for replacing invalid code units
- Dropping illegal code units
- Producing an error log message
- Collecting statistics on transcoding errors
- Implementing a custom transcoding view whose `value_type` is `std::expected<charN_t, std::uc::transcoding_error>`

§ 5.3.1 Why `std::expected<void, E>`?

The main alternative to consider here would be to specify that default-constructed `std::uc::transcoding_error` values represent success, or add a success enumerator whose value is zero. There is precedent for doing this in the standard in the error handling approach of `std::from_chars`, which returns a `std::from_chars_result` containing a `std::errc` that has an `operator bool()` that returns `true` if the `std::errc` is default-constructed.

However, that design decision was made before `std::expected` was added to the standard library in C++23. Now that we have this facility, we should take the opportunity to use the type system to structurally separate the error cases from the success cases, instead of lumping them all together in the same type as in the case of `std::errc`.

§ 5.3.2 Existing practice

- `iconv()`
 - Uses two possible `errno` error codes:
 - `EINVAL` (the initial subsequence of a valid sequence was at the end of the input sequence)
 - `EILSEQ` (any other invalid sequence in the input)
 - Uses an out-parameter to point to the beginning of the invalid sequence
- ICU
 - `u_strFromUTF8WithSub()`

- Either replaces invalid input sequences with a user-provided `int32_t` (documentation recommends U+FFFD) or sets an error code using an out-param
- `MultiByteToWideChar()`
 - User specifies a flag to decide whether to fail on invalid input
 - If set, any invalid input results in the error code `ERROR_NO_UNICODE_TRANSLATION`
 - If unset, uses U+FFFD replacement (unless pre-Vista in which case illegal sequences are dropped)
- Python `decode()`
 - Invalid sequences result in exceptions containing verbose descriptions of the offending sequence
 - Example:

```
>>> b'\x80abc'.decode("utf-8", "strict")
Traceback (most recent call last):
...
UnicodeDecodeError: 'utf-8' codec can't decode byte 0x80 in position 0:
invalid start byte
```
 - List of error messages:
 - unexpected end of data
 - invalid start byte
 - invalid continuation byte
 - code point in surrogate code point range(`0xd800`, `0xe000`)
 - truncated data
 - code point `not` in range(`0x110000`)
 - illegal encoding
 - illegal UTF-16 surrogate

The claim that `to_utfN_view`'s `success()` API is a basis operation is supported by the fact that each of the above APIs can be implemented using it, but not vice versa. See [Appendix: Implementing Existing Practice for Error Handling](#) for code examples which demonstrate this.

§ 5.3.3 `std::uc::transcoding_error` enumerators

- `truncated_utf8_sequence`
 - An ill-formed subsequence that matches the beginning of some well-formed sequence.
 - Example invalid code unit sequence: UTF-8 `0xE1 0x80`.
- `unpaired_high_surrogate`
 - Example invalid code unit sequence: UTF-16 `0xD800`.
- `unpaired_low_surrogate`
 - Example invalid code unit sequence: UTF-16 `0xDC00`.
- `unexpected_utf8_continuation_byte`
 - Example invalid code unit sequence: UTF-8 `0x80`.

- `overlong`
 - An overlong UTF-8 encoding.
 - Example invalid code unit sequence: UTF-8 `0xE0 0x80`.
- `encoded_surrogate`
 - Applies to both UTF-8 and UTF-32.
 - Example invalid code unit sequence: UTF-8 `0xED 0xA0`, UTF-32 `0x0000D800`.
- `out_of_range`
 - Applies to both UTF-8 and UTF-32
 - In UTF-8, this applies to `0xF4` if it is followed by a continuation byte greater than `0x8F`
 - In UTF-32, this is any code unit greater than `0x10FFFF`
 - Example invalid code unit sequence: UTF-8 `0xF4 0x90`, UTF-32 `0x110000`.
- `invalid_utf8_leading_byte`
 - In UTF-8, this applies to `0xC0-0xC1` and `0xF5-0xFF`.
 - Example invalid code unit sequence: UTF-8 `0xC0`.

An alternative approach to minimize the number of enumerators could merge `truncated_utf8_sequence` with `unpaired_high_surrogate` and merge `unexpected_utf8_continuation_byte` with `unpaired_low_surrogate`, but based on feedback, splitting these up seems to be preferred.

§ 5.3.4 Examples

The first two rows of each of the following tables are taken directly from the “U+FFFD Substitution of Maximal Subparts” section of the Unicode standard, and augmented to show the associated `success()` for each resulting code point.

Table 3-8. U+FFFD for Non-Shortest Form Sequences

Bytes	C0	AF	E0	80	BF	F0	81	82	41
Output	FFFD	FFFD	FFFD	FFFD	FFFD	FFFD	FFFD	FFFD	0041
error()	invalid_utf8_leading_byte	unexpected_utf8_continuation_byte	overlong	unexpected_utf8_continuation_byte	unexpected_utf8_continuation_byte	overlong	unexpected_utf8_continuation_byte	unexpected_utf8_continuation_byte	

Table 3-9. U+FFFD for Ill-Formed Sequences for Surrogates

Bytes	E0	A0	80	ED	BF	BF	ED	AF	41
Output	FFFD	FFFD	FFFD	FFFD	FFFD	FFFD	FFFD	FFFD	0041
error()	encoded_surrogate	unexpected_utf8_continuation_byte	unexpected_utf8_continuation_byte	encoded_surrogate	unexpected_utf8_continuation_byte	unexpected_utf8_continuation_byte	encoded_surrogate	unexpected_utf8_continuation_byte	

Table 3-10. U+FFFD for Other Ill-Formed Sequences

Bytes	F4	91	92	93	FF	41	80	BF	42
Output	FFFD	FFFD	FFFD	FFFD	FFFD	FFFD	FFFD	FFFD	0041
error()	out_of_range	unexpected_utf8_continuation_byte	unexpected_utf8_continuation_byte	unexpected_utf8_continuation_byte	invalid_utf8_leading_byte		unexpected_utf8_continuation_byte	unexpected_utf8_continuation_byte	

Table 3-11. U+FFFD for Truncated Sequences

Bytes	E1	80	E2	F0	91	92	F1	BF	41
Output	FFFD		FFFD	FFFD			FFFD		0041
error()	truncated_utf8_sequence		truncated_utf8_sequence	truncated_utf8_sequence			truncated_utf8_sequence		

Note that outside of the truncation case, the leading code unit is associated with a more specific error enumerator, and then all the continuation bytes in the invalid sequence are `unexpected_utf8_continuation_byte`. This is aligned with my interpretation of the underlying logic of Substitution of Maximal Subparts; also, any other approach would require additional lookahead, which would break some of the API's invariants.

§ 5.4 Erroneous Behavior

Iterators are constructed from more than one underlying iterator. In order to perform iteration in many text-handling contexts, you need to know the beginning and the end of the range you are iterating over, just to be able to perform iteration correctly. Note that this is not a safety issue, but a correctness one. For example, say we have a string `s` of UTF-8 code units that we would like to iterate over to produce UTF-32 code points. If the last code unit in `s` is `0xe0`, we should expect two more code units to follow. They are not present, though, because `0xe0` is the last code unit. Now consider how you would implement `operator++()` for an iterator `iter` that transcodes from UTF-8 to UTF-32. If you advance far enough to get the next UTF-32 code point in each call to `operator++()`, you may run off the end of `s` when you find `0xe0` and try to read two more code units. Note that it does not matter that `iter` probably comes from a range with an end-iterator or sentinel as its mate; inside `iter`'s `operator++()` this is no help. `iter` must therefore have the end-iterator or sentinel as a data member. The same logic applies to the other end of the range if `iter` is bidirectional — it must also have the iterator to the start of the underlying range as a data member. This unfortunate reality comes up over and over in the proposed iterators, not just the ones that are UTF transcoding iterators. This is why iterators in this proposal (and the ones to come) usually consist of three underlying iterators.

Because of this fact, it's almost free to specify these iterators so that dereferencing a past-the-end iterator, incrementing a past-the-end iterator, and decrementing an at-the-beginning iterator are all erroneous behavior instead of undefined behavior. The only time an additional branch is required to

ensure safety is to check for a before-the-beginning decrement in `operator--` (although actually producing diagnostics for the EB requires further branching).

As long as a transcoding view is constructed with proper arguments, all subsequent operations on it and its iterators are memory safe.

§ 5.5 Optimization for transcoding views wrapping other transcoding views

In generic contexts, users will create `to_utfN_views` wrapping iterators of other `to_utfN_views`. This presents a problem for a naive implementation because when `to_utfN_view` is wrapping a bidirectional range, the number of iterators in each successive `to_utfN_view` wrapper increases geometrically unless we use workarounds.

The workaround makes it so that when a `to_utfN_view` is constructed from another `to_utfN_view`'s iterators, instead of storing those iterators in the iterators of the outer `to_utfN_view`, the outer `to_utfN_view`'s iterators have identical contents to the inner `to_utfN_view`'s iterators, the only difference being the output encoding. This also allows the outer `to_utfN_view`'s iterators to reconstruct the inner `to_utfN_view` iterator when its `base()` member function is invoked, without actually storing it.

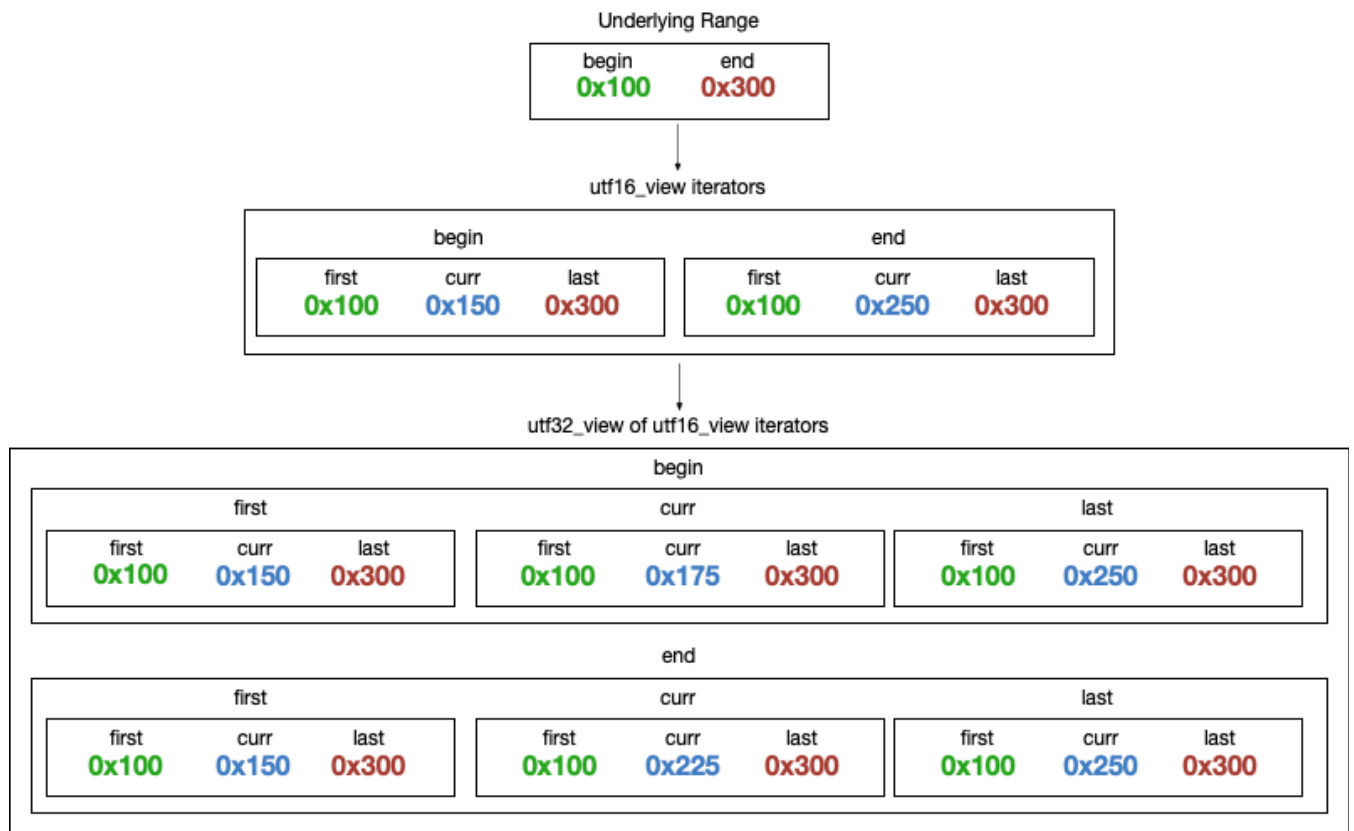
This optimization is only needed when the underlying range is bidirectional (or “better”), because input ranges and forward ranges increase in size linearly rather than geometrically with each successive wrapper, due to the fact that the sentinel is not wrapped by the transcoding iterator.

Although it's not strictly necessary, we could also apply the optimization when the underlying range is a forward range, preventing the iterator size from growing at all (as opposed to linear growth), but that isn't done in this paper because we judge the tradeoffs as not being justified. It is not possible to apply the optimization when the underlying range is an input range, because of the fact that the underlying iterator is past-the-end of the current code point.

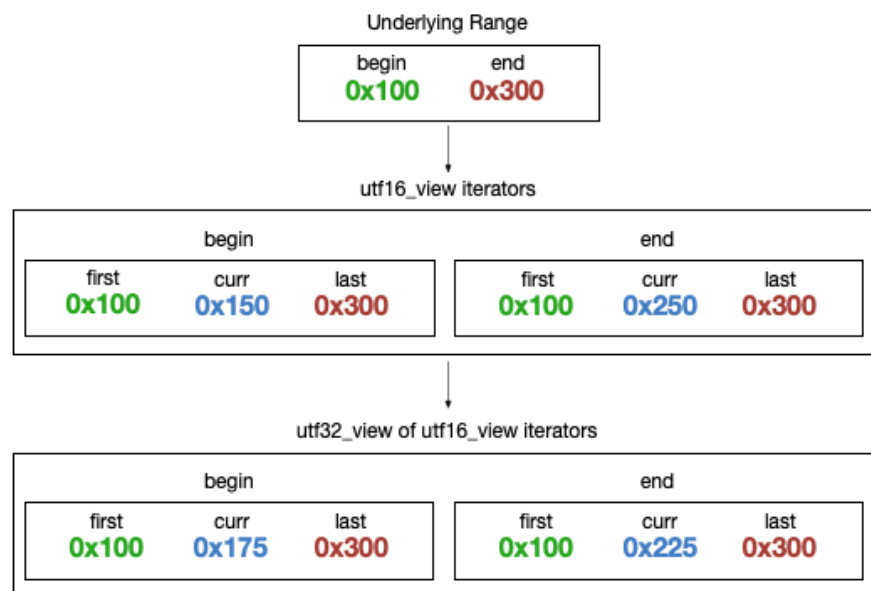
The diagram below represents the outcome of the following process:

- The user starts with a range of `char8_ts` from `0x100` to `0x300`.
- They create a `to_utf16_view` with this underlying range.
- They advance that view's `begin()` iterator until the underlying pointer is at `0x150` and reverse the view's `end()` iterator until the underlying pointer is at `0x250`.
- They create a subrange with these iterators and wrap it in a `to_utf32_view`.
- They advance the `to_utf32_view`'s `begin()` iterator until the underlying pointer (two levels down) is at `0x175` and similarly reverse `end()` to `0x225`.

Naive Implementation



Optimized



The goal is for the optimized implementation to avoid having to store all the iterators that the naive implementation does, while still outwardly appearing to the user as though its API is the same as the naive one.

The iterators of the optimized `to_utf32_view` can simulate the naive version's `base()` by reconstructing a `to_utf16_view` iterator containing its own `first`, `curr`, and `last` iterators. However, if we added accessors for `first()` and `last()` to the iterator, then we wouldn't be able to return the same results as the naive implementation because we've lost information about those iterators— so this optimization can only work properly if we leave those out.

Unlike with other range adaptor objects, `base()` cannot have any overloads that simply return a reference to the underlying iterator as opposed to a new copy or move-constructed instantiation of it, because of this optimization.

Input iterators cannot benefit from this optimization because they are necessarily past-the-end of the current code point within the range being adapted, whereas other iterator types are at the beginning of the current code point.

There is an unavoidable inconsistency introduced by this optimization that occurs when one of the iterators is in the middle of a code point in a variable length encoding (UTF-8 or UTF-16). Consider what happens when a user attempts to convert the UTF-32 code point U+1F574  MAN IN BUSINESS SUIT LEVITATING to UTF-8 and then to UTF-16, but increments the iterator of the UTF-8 transcoding view by one code unit first.

In the naive implementation, the result is simply three replacement characters as the UTF-16 transcoder encounters three unexpected UTF-8 continuation bytes:

```
UTF-32: 0x1F574
UTF-8:  0xF0    0x9F    0x95    0xB4
          ^
UTF-16:      0xFFFD 0xFFFD 0xFFFD
```

However, in the optimized implementation, when the UTF-16 transcoding view wraps the iterator from the UTF-8 transcoding view, it looks directly at the underlying UTF-32 iterator and forgets the UTF-8 iterator's position within the code point:

```
UTF-32: 0x1F574
UTF-16: 0xD83D 0xDD74
```

Furthermore, when you invoke `base()` on an iterator of the UTF-16 transcoding view, it's lost the intra-code-point position, moving it back to the starting code unit:

```
Original iterator:
UTF-8:  0xF0    0x9F    0x95    0xB4
          ^

Result of base():
UTF-8:  0xF0    0x9F    0x95    0xB4
          ^
```

These inconsistencies are somewhat unfortunate, but they only apply when the input to the transcoding view starts in the middle of a code point, which is definitionally invalid UTF anyway; and it does not affect the invariant that the output is always valid UTF. This is an acceptable tradeoff for avoiding quadratic growth of the iterator sizes.

There is one more quirk introduced by this optimization. For ordinary, non-special-cased iterators of transcoding views, dereferencing a past-the-end iterator, incrementing past the end, and decrementing before the beginning are all erroneous behavior. However, because of the information loss associated with this optimization, the EB detection can't kick in until the user has exceeded the bounds of the deepest underlying range, rather than one of its intermediate layers.

For example, in the scenario in the diagram from before, the naive implementation would detect EB when the `to_utf32_view`'s `begin()` iterator was decremented to the point where the underlying range iterator was less than `0x150`, but the special-cased implementation would simply continue reading through the underlying range until `0x100`. This is perhaps surprising, but still achieves memory safety.

It's a useful property of this approach that the type system remembers the correct type to use for `base()` even in the case of transcoding views wrapping other transcoding view. To illustrate, consider this algorithm (not proposed) as an example.

```
template<input_iterator I, sentinel_for<I> S, output_iterator<char8_t> O>
transcode_result<I, O> transcode_to_utf32(I first, S last, O out);
```

Such a transcoding algorithm is pretty similar to `std::ranges::copy`, in that you should return both the output iterator *and* the final position of the input iterator (`transcode_result` is an alias for `in_out_result`). Because we can always provide `base()`, we have no trouble returning a `transcode_result` here in every case:

```
template<input_iterator I, sentinel_for<I> S, output_iterator<char8_t> O>
transcode_result<I, O> transcode_to_utf32(I first, S last, O out) {
    auto r = ranges::subrange(first, last) | uc::as_utf32;

    auto copy_result = ranges::copy(r, out);

    return transcode_result<I, O>{copy_result.in.base(), copy_result.out};
}
```

§ 5.6 Other design notes

None of the proposed interfaces is subject to change in future versions of Unicode; each relates to the guaranteed-stable subset. Just sayin'.

None of the proposed interfaces allocates or throws.

All the transcoding iterators allow you access to the underlying iterator via `.base()`, following the convention of the iterator adaptors already in the standard.

The transcoding views are lazy, as you'd expect. They also compose with the standard view adaptors, so just transcoding at most 10 UTF-16 code units out of some UTF can be done with `foo | std::uc::to_utf16 | std::ranges::views::take(10)`.

Error handling strategies of the user's choosing can be implemented by the user due to the suitable basis operation `success()` provided by the transcoding iterator. This gives control to those who want to do something other than the default. The default, according to Unicode, is to produce a replacement character (`0xfffd`) in the output when broken UTF encoding is seen in the input. This is what all these interfaces do, unless you make use of the basis operation.

The production of replacement characters as error-handling strategy is good for memory compactness and safety. It allows us to store all our text as UTF-8 (or, less compactly, as UTF-16), and then process code points as transcoding views. If an error occurs, the transcoding views will simply produce a replacement character; there is no danger of UB.

§ 5.7 Null-terminated sequence sentinel `null_sentinel` and associated CPO `null_term`

```
namespace std {

    template<class I>
    concept default-initializable-and-equality-comparable-iter-value =
        default_initializable<iter_value_t<I>> &&
        equality_comparable_with<iter_reference_t<I>, iter_value_t<I>>; // exposition
                                only

}
```



```

struct null_sentinel_t {
    template<input_iterator I>
        requires (not forward_iterator<I>) && default-initializable-and-equality-comparable-iter-value<I>
    friend constexpr bool operator==(I const& it, null_sentinel_t) {
        return *it == iter_value_t<I>{};
    }
    template<forward_iterator I>
        requires default-initializable-and-equality-comparable-iter-value<I>
    friend constexpr bool operator==(I it, null_sentinel_t) {
        return *it == iter_value_t<I>{};
    }
};

inline constexpr null_sentinel_t null_sentinel;

inline constexpr unspecified null_term;
}

```

The sentinel type matches any iterator position `it` at which `*it` is equal to a default-constructed object of type `iter_value_t<I>`. This works for null-terminated strings, but can also serve as the sentinel for any range terminated by a default-constructed value.

Because this type is potentially useful for lots of ranges unrelated to Unicode or text, it is in the `std` namespace, not `std::uc`.

The `null_sentinel_t`'s `operator==` has a separate overload for input iterators that takes the iterator by reference instead of by value. We want to take input iterators by reference because they are not required to be copyable. However, for forward iterators, we want to take by value because otherwise we incur a double indirection (e.g. `int* const& it`) that compilers may not optimize.

The name `null_term` denotes a customization point object ([customization.point.object]). Given a subexpression `E`, the expression `null_term(E)` is expression-equivalent to `ranges::subrange(move(E), null_sentinel)`.

§ 5.8 Exposition-only concepts and traits

```

namespace std::uc {

    template<class T>
    constexpr bool is-empty-view = false;
    template<class T>
    constexpr bool is-empty-view<ranges::empty_view<T>> = true;

    template<class T>
    concept code-unit-to = same_as<remove_cv_t<T>, char8_t> ||
        same_as<remove_cv_t<T>, char16_t> || same_as<remove_cv_t<T>, char32_t>;

    template<class T>
    concept code-unit-from =
        same_as<remove_cv_t<T>, char> || same_as<remove_cv_t<T>, wchar_t> || code-unit-to<T>;

    template<class T>
    concept utf-range =
        ranges::input_range<T> && code-unit-from<ranges::range_value_t<T>>;
}

```

```

template<class I>
constexpr auto bidirectional-at-most() { // exposition only
    if constexpr (bidirectional_iterator<I>) {
        return bidirectional_iterator_tag{};
    } else if constexpr (forward_iterator<I>) {
        return forward_iterator_tag{};
    } else if constexpr (input_iterator<I>) {
        return input_iterator_tag{};
    }
}

template<class I>
using bidirectional-at-most-t = decltype(bidirectional-at-most<I>()); // exposition only

template<class I>
constexpr auto iterator-to-tag() { // exposition only
    if constexpr (random_access_iterator<I>) {
        return random_access_iterator_tag{};
    } else if constexpr (bidirectional_iterator<I>) {
        return bidirectional_iterator_tag{};
    } else if constexpr (forward_iterator<I>) {
        return forward_iterator_tag{};
    } else if constexpr (input_iterator<I>) {
        return input_iterator_tag{};
    }
}

template<class I>
using iterator-to-tag-t = decltype(iterator-to-tag<I>()); // exposition only
}

```

§ 5.9 Transcoding views

```

namespace std::uc {

enum class transcoding_error {
    truncated_utf8_sequence,
    unpaired_high_surrogate,
    unpaired_low_surrogate,
    unexpected_utf8_continuation_byte,
    overlong,
    encoded_surrogate,
    out_of_range,
    invalid_utf8_leading_byte
};

template<class T>
concept to-utf-view-iterator-optimizable = unspecified // exposition only

template<code-unit-to ToType, from-utf-view V>
class to-utf-view-impl : public ranges::view_interface<to-utf-view-impl<ToType, V>>
{
public:
    template<bool Const>
        class utf-iterator : public iterator_interface<bidirectional-at-most-t
            <ranges::iterator_t<V>>, ToType, ToType> {

private:
    using iter = ranges::iterator_t<maybe-const<Const, V>>;
    using sent = ranges::sentinel_t<maybe-const<Const, V>>;

```

```

template<code-unit-to ToType2,
        from-utf-view V2>
friend class to-utf-view-impl; // exposition only

template<class I>
struct first-and-curr { // exposition only
    first-and-curr() = default;
    constexpr first-and-curr(I curr) : curr(move(curr)) {}

    I curr;
};

template<bidirectional_iterator I>
struct first-and-curr<I> { // exposition only
    first-and-curr() = default;
    constexpr first-and-curr(I first, I curr) : first(first), curr(curr) {}

    I first;
    I curr;
};

using innermost-iter = unspecified; // exposition only

using from-type = decltype([] {
    if constexpr (is_same_v<char, iter_value_t<innermost-iter>>) {
        return char8_t{};
    } else if constexpr (is_same_v<wchar_t, iter_value_t<innermost-iter>>) {
        if constexpr (sizeof(wchar_t) == 2) {
            return char16_t{};
        } else if constexpr (sizeof(wchar_t) == 4) {
            return char32_t{};
        }
    } else {
        return iter_value_t<innermost-iter>{};
    }
})(); // exposition only

using innermost-iter = unspecified; // exposition only
using innermost-sent = unspecified; // exposition only

public:
    using value_type = ToType;
    using reference_type = ToType&;
    using difference_type = ptrdiff_t;
    using iterator_concept = bidirectional-at-most-t<iter>;

    constexpr utf-iterator() requires default_initializable<V> = default;

private:
    constexpr utf-iterator(innermost-iter first, innermost-iter it, innermost-sent
        last) // exposition only
        requires bidirectional_iterator<innermost-iter>
        : first_and_curr_(first, it), last_(last) {
        if (curr() != last_)
            read();
    }

    constexpr utf-iterator(innermost-iter it, innermost-sent last) // exposition
        only
        requires (!bidirectional_iterator<innermost-iter>)
        : first_and_curr_(move(it)), last_(last) {
        if (curr() != last_)
            read();
    }

```

```

    }

public:
    constexpr utf_iterator() = default;
    constexpr utf_iterator(utf_iterator const&) requires copyable<innermost_iter> =
        default;

        constexpr utf_iterator& operator=(utf_iterator const&) requires
            copyable<innermost_iter> = default;

    constexpr utf_iterator(utf_iterator&&) = default;

    constexpr utf_iterator& operator=(utf_iterator&&) = default;

    constexpr iter base() const requires forward_iterator<innermost_iter>
    {
        if constexpr (to_utf_view_iterator_optimizable<iter>) {
            if constexpr (bidirectional_iterator<innermost_iter>) {
                return iter(begin(), curr(), last_);
            } else {
                return iter(curr(), last_);
            }
        } else {
            return curr();
        }
    }

    constexpr iter base() &&
        requires (!forward_iterator<innermost_iter>) { return move(*this).curr(); }

    constexpr expected<void, transcoding_error> success() const;

    constexpr value_type operator*() const;

    constexpr utf_iterator& operator++() {
        if constexpr (forward_iterator<innermost_iter>) {
            if (buf_index_ + 1 < buf_last_) {
                ++buf_index_;
            } else if (buf_index_ + 1 == buf_last_) {
                advance(curr(), to_increment_);
                to_increment_ = 0;
                if (curr() != last_) {
                    read();
                } else {
                    buf_index_ = 0;
                }
            }
        } else {
            if (buf_index_ + 1 == buf_last_ && curr() != last_) {
                read();
            } else if (buf_index_ + 1 <= buf_last_) {
                ++buf_index_;
            }
        }
        return *this;
    }

    constexpr auto operator++(int) {
        if constexpr (is_same_v<iterator_concept, input_iterator_tag>) {
            ++*this;
        } else {
            auto retval = *this;

```

```

        ++*this;
        return retval;
    }
}

constexpr utf-iterator& operator--() requires bidirectional_iterator<innermost-iter>
{
    if (!buf_index_)
        read_reverse();
    else if (buf_index_)
        --buf_index_;
    return *this;
}

constexpr utf-iterator operator--(int) requires
bidirectional_iterator<innermost-iter>
{
    auto retval = *this;
    --*this;
    return retval;
}

friend constexpr bool operator==(utf-iterator const& lhs, utf-iterator const&
rhs)
requires forward_iterator<innermost-iter> || requires (innermost-iter i) { i
!= i; }
{
    if constexpr (forward_iterator<innermost-iter>) {
        return lhs.curr() == rhs.curr() && lhs.buf_index_ == rhs.buf_index_;
    } else {
        if (lhs.curr() != rhs.curr())
            return false;

        if (lhs.buf_index_ == rhs.buf_index_ && lhs.buf_last_ == rhs.buf_last_) {
            return true;
        }

        return lhs.buf_index_ == lhs.buf_last_ && rhs.buf_index_ == rhs.buf_last_;
    }
}

friend constexpr bool operator==(utf-iterator const& lhs, innermost-sent rhs)
requires copyable<innermost-iter>
{
    if constexpr (forward_iterator<innermost-iter>) {
        return lhs.curr() == rhs;
    } else {
        return lhs.curr() == rhs && lhs.buf_index_ == lhs.buf_last_;
    }
}

friend constexpr bool operator==(utf-iterator const& lhs, innermost-sent rhs)
requires (!copyable<innermost-iter>)
{
    return lhs.curr() == rhs && lhs.buf_index_ == lhs.buf_last_;
}

constexpr innermost-iter begin() const // exposition only
requires bidirectional_iterator<innermost-iter>
{
    return first_and_curr_.first;
}

```

```

    }
    constexpr innermost-sent end() const { // exposition only
        return last_;
    }

    constexpr void read(); // exposition only

    constexpr void read_reverse(); // exposition only

        constexpr innermost-iter& curr() & { return first_and_curr_.curr; } //
        exposition only

    constexpr innermost-iter const& curr() const& { return first_and_curr_.curr; }
        // exposition only

    constexpr innermost-iter curr() && { return move(first_and_curr_.curr); } //
        exposition only

    array<value_type, 4 / sizeof(ToType)> buf_{}; // exposition only

    first-and-curr<innermost-iter> first_and_curr_; // exposition only

    [[no_unique_address]] innermost-sent last_; // exposition only

    uint8_t buf_index_ = 0; // exposition only
    uint8_t buf_last_ = 0; // exposition only
    uint8_t to_increment_ = 0; // exposition only
};

private:
    template<bool Const>
    static constexpr auto make_begin(auto first, auto last) { // exposition only
        if constexpr (bidirectional_iterator<ranges::iterator_t<V>>) {
            if constexpr (to-utf-view-iterator-optimizable<ranges::iterator_t<V>>) {
                return utf-iterator<Const>(first.begin(), first.curr(), first.last_);
            } else {
                return utf-iterator<Const>(first, first, last);
            }
        } else {
            return utf-iterator<Const>(move(first), last);
        }
    }
    template<bool Const>
    static constexpr auto make_end(auto first, auto last) { // exposition only
        if constexpr (bidirectional_iterator<ranges::sentinel_t<V>>) {
            if constexpr (to-utf-view-iterator-optimizable<ranges::sentinel_t<V>>) {
                return utf-iterator<Const>(last.begin(), last.curr(), last.last_);
            } else {
                return utf-iterator<Const>(first, last, last);
            }
        } else {
            return last;
        }
    }

    V base_ = V(); // exposition only

public:
    constexpr to-utf-view-impl() requires default_initializable<V> = default;
    constexpr to-utf-view-impl(V base) : base_(move(base)) {}

    constexpr V base() const& requires copy_constructible<V>

```

```

{
    return base_;
}
constexpr V base() && { return move(base_); }

constexpr auto begin() requires (!copyable<ranges::iterator_t<V>>)
{
    return make_begin<false>(ranges::begin(base_), ranges::end(base_));
}
constexpr auto begin() const requires copyable<ranges::iterator_t<V>>
{
    return make_begin<true>(ranges::begin(base_), ranges::end(base_));
}

constexpr auto end() requires (!copyable<ranges::iterator_t<V>>)
{
    return make_end<false>(ranges::begin(base_), ranges::end(base_));
}
constexpr auto end() const requires copyable<ranges::iterator_t<V>>
{
    return make_end<true>(ranges::begin(base_), ranges::end(base_));
}

constexpr bool empty() const { return ranges::empty(base_); }
};

template<from-utf-view V>
class to_utf8_view {
private:
    using iterator = ranges::iterator_t<to-utf-view-impl<char8_t, V>>;
    using sentinel = ranges::sentinel_t<to-utf-view-impl<char8_t, V>>;

public:
    constexpr to_utf8_view() requires default_initializable<V> = default;
    constexpr to_utf8_view(V base) : impl_(move(base)) {}

    constexpr V base() const& requires copy_constructible<V>
    {
        return impl_.base();
    }
    constexpr V base() && { return move(impl_.base()); }

    constexpr auto begin() requires (!copyable<iterator>)
    {
        return impl_.begin();
    }
    constexpr auto begin() const requires copyable<iterator>
    {
        return impl_.begin();
    }

    constexpr auto end() requires (!copyable<iterator>)
    {
        return impl_.end();
    }
    constexpr auto end() const requires copyable<iterator>
    {
        return impl_.end();
    }

    constexpr bool empty() const { return impl_.empty(); }

```

```

private:
    to-utf-view-impl<char8_t, V> impl_;
};

template<class R>
to_utf8_view(R&&) -> to_utf8_view<views::all_t<R>>;

template<from-utf-view V>
class to_utf16_view {
private:
    using iterator = ranges::iterator_t<to-utf-view-impl<char16_t, V>>;
    using sentinel = ranges::sentinel_t<to-utf-view-impl<char16_t, V>>;

public:
    constexpr to_utf16_view() requires default_initializable<V> = default;
    constexpr to_utf16_view(V base) : impl_(move(base)) {}

    constexpr V base() const& requires copy_constructible<V>
    {
        return impl_.base();
    }
    constexpr V base() && { return move(impl_.base()); }

    constexpr auto begin() requires (!copyable<iterator>)
    {
        return impl_.begin();
    }
    constexpr auto begin() const requires copyable<iterator>
    {
        return impl_.begin();
    }

    constexpr auto end() requires (!copyable<iterator>)
    {
        return impl_.end();
    }
    constexpr auto end() const requires copyable<iterator>
    {
        return impl_.end();
    }

    constexpr bool empty() const { return impl_.empty(); }

private:
    to-utf-view-impl<char16_t, V> impl_;
};

template<class R>
to_utf16_view(R&&) -> to_utf16_view<views::all_t<R>>;

template<from-utf-view V>
class to_utf32_view {
private:
    using iterator = ranges::iterator_t<to-utf-view-impl<char32_t, V>>;
    using sentinel = ranges::sentinel_t<to-utf-view-impl<char32_t, V>>;

public:
    constexpr to_utf32_view() requires default_initializable<V> = default;
    constexpr to_utf32_view(V base) : impl_(move(base)) {}

    constexpr V base() const& requires copy_constructible<V>
    {

```



```

    return impl_.base();
}
constexpr V base() && { return move(impl_).base(); }

constexpr auto begin() requires (!copyable<iterator>)
{
    return impl_.begin();
}
constexpr auto begin() const requires copyable<iterator>
{
    return impl_.begin();
}

constexpr auto end() requires (!copyable<iterator>)
{
    return impl_.end();
}
constexpr auto end() const requires copyable<iterator>
{
    return impl_.end();
}

constexpr bool empty() const { return impl_.empty(); }

private:
    to-utf-view-impl<char32_t, V> impl_;
};

template<class R>
to_utf32_view(R&&) -> to_utf32_view<views::all_t<R>>;

template<code-unit-to ToType>
inline constexpr unspecified to_utf;

inline constexpr unspecified to_utf8;

inline constexpr unspecified to_utf16;

inline constexpr unspecified to_utf32;
}

namespace std::ranges {

    template <class ToType, class V>
        inline constexpr bool enable_borrowed_range<
            std::uc::to-utf-view-impl<ToType, V>> = enable_borrowed_range<V>;

    template<class V>
        inline constexpr bool enable_borrowed_range<std::uc::to_utf8_view<V>> =
            enable_borrowed_range<V>;

    template<class V>
        inline constexpr bool enable_borrowed_range<std::uc::to_utf16_view<V>> =
            enable_borrowed_range<V>;

    template<class V>
        inline constexpr bool enable_borrowed_range<std::uc::to_utf32_view<V>> =
            enable_borrowed_range<V>;

}

```

The exposition-only concept *to-utf-view-iterator-optimizable* is true if its template parameter is a specialization of *utf-iterator* and it is a *std::ranges::bidirectional_iterator*.

to-utf-view-impl is an exposition-only class that provides implementation details common to the three transcoding views, *to_utf8_view*, *to_utf16_view*, and *to_utf32_view*, which are themselves described further down.

The iterator type of *to-utf-view-impl* is *utf-iterator*. *utf-iterator* is an iterator that transcodes from UTF-N to UTF-M, where N and M are each one of 8, 16, or 32. N may equal M.

utf-iterator uses a mapping between character types and UTF encodings, which is that that *char* and *char8_t* correspond to UTF-8, *char16_t* corresponds to UTF-16, *char32_t* corresponds to UTF-32, and *wchar_t* corresponds to UTF-16 if its size is two or UTF-32 if its size is 4.

utf-iterator does its work by adapting an underlying range of code units. We use the term “input subsequence” to refer to a potentially ill-formed code unit subsequence which is to be transcoded into a code point *c*. Each input subsequence is decoded from the UTF encoding corresponding to *from-type*. If the underlying range contains ill-formed UTF, the code units are divided into input subsequences according to Substitution of Maximal Subparts, and each ill-formed input subsequence is transcoded into a U+FFFD. *c* is then encoded to *ToType*’s corresponding encoding, into an internal code unit buffer.

utf-iterator maintains certain invariants; the invariants differ based on whether *utf-iterator* is an input iterator.

For input iterators the invariant is: if **this* is at the end of the range being adapted, then *curr()* == *last_*; otherwise, the position of *curr()* is always at the end of the input subsequence corresponding to the current code point *c*, and *buf_* contains the code units that comprise *c*, in the UTF encoding corresponding to *ToType*.

For forward and bidirectional iterators, the invariant is: if **this* is at the end of the range being adapted, then *curr()* == *last_*; otherwise, the position of *curr()* is always at the beginning of the input subsequence corresponding to the current code point *c* within the underlying range, and *buf_* contains the code units in *ToFormat* that comprise *c*.

The exposition-only member function *read* decodes the input subsequence starting at position *curr()* into a code point *c*, using the UTF encoding corresponding to *from-type*, and setting *c* to U+FFFD if the input subsequence is ill-formed. If *c* is set to U+FFFD as the result of an ill-formed input subsequence, it sets the error as described below. It sets *to_increment_* to the number of code units read while decoding *c*; encodes *c* into *buf_* in the UTF encoding corresponding to *ToType*; sets *buf_index_* to 0; and sets *buf_last_* to the number of code units encoded into *buf_*. If *forward_iterator<I>* is *true*, *curr()* is set to the position it had before *read* was called.

The exposition-only member function *read_reverse* decodes the input subsequence ending at position *curr()* into a code point *c*, using the UTF encoding corresponding to *from-type*, and setting *c* to U+FFFD if the input subsequence is ill-formed. If *c* is set to U+FFFD as the result of an ill-formed input subsequence, it sets the error as described below. It sets *to_increment_* to the number of code units read while decoding *c*; encodes *c* into *buf_* in the UTF encoding corresponding to *ToType*; sets *buf_last_* to the number of code units encoded into *buf_*; and sets *buf_index_* to *buf_last_ - 1*.

In the following paragraph, *utf-error(foo)* refers to the result of the exposition-only function:

```
expected<void, transcoding_error> utf-error-func(transcoding_error err) {  
    return unexpected{err};  
}
```

When the *utf-iterator* is at the end of the underlying range, `success()` returns a default-constructed `expected<void, transcoding_error>`. When the *utf-iterator* has a code unit, derived from a code point *c*, which is itself derived from a particular input subsequence (the “current input subsequence”), the result of the `success()` method corresponds to the underlying range’s input subsequences as follows. (All ranges of numerical values of code units below are inclusive.)

- If the encoding corresponding to *from-type* is UTF-8:
 - If the current input subsequence is valid UTF-8, `success()` returns `expected<void, transcoding_error>{}`.
 - If the current input subsequence is a code unit between 0x80 and 0xBF, `success()` returns `utf_error(transcoding_error::unexpected_utf8_continuation_byte)`.
 - If the current input subsequence is a code unit between 0xC0 and 0xC2, or between 0xF5 and 0xFF, `success()` returns `utf_error(transcoding_error::invalid_utf8_leading_byte)`.
 - If the current input subsequence is 0xE0, and the subsequent input subsequence is between 0x80 and 0x9F; or if the current input subsequence is 0xF0, and the subsequent input subsequence is between 0x80 and 0x8F; then `success()` returns `utf_error(transcoding_error::overlong)`.
 - If the current input subsequence is 0xED, and the subsequent input subsequence is between 0xA0 and 0xBF, then `success()` returns `utf_error(transcoding_error::encoded_surrogate)`.
 - If the the current input subsequence is 0xF4, and the subsequent input subsequence is between 0x90 and 0xBF, then `success()` returns `utf_error(transcoding_error::out_of_range)`.
 - Otherwise, if the current input subsequence is invalid UTF-8, begins with a code unit between 0xC2 and 0xF4, and there exists some hypothetical sequence of code units which would make the current input subsequence well-formed if concatenated to the end of it, `success()` returns `utf_error(transcoding_error::truncated_utf8_sequence)`.
- If the encoding corresponding to *from-type* is UTF-16:
 - If the current input subsequence is valid UTF-16, `success()` returns `expected<void, transcoding_error>{}`.
 - If the current input subsequence is between 0xD800 and 0xDBFF, `success()` returns `utf_error(transcoding_error::unpaired_high_surrogate)`.
 - If the current input subsequence is between 0xDC00 and 0xDFFF, `success()` returns `utf_error(transcoding_error::unpaired_low_surrogate)`.
- If the encoding corresponding to *from-type* is UTF-32:
 - If the current input subsequence is valid UTF-32, `success()` returns `expected<void, transcoding_error>{}`.
 - If the current input subsequence is between 0xD800 and 0xDFFF, `success()` returns `utf_error(transcoding_error::encoded_surrogate)`.
 - If the current input subsequence is between 0x110000 and 0xFFFFFFFF, `success()` returns `utf_error(transcoding_error::out_of_range)`.

utf-iterator’s exposition-only type alias *innermost-iter* is `iter::innermost-iter` if *iter* is `to_utf_view_iterator_optimizable`, or *iter* otherwise. The exposition-only type alias *innermost-sent* is `sent::innermost-sent` if *sent* is `to_utf_view_iterator_optimizable`, or *sent* otherwise.

If *utf-iterator* is a `bidirectional_iterator`, it is defined to be at the beginning of its underlying range if `buf_index_` is zero and `curr() == begin()`. If it is a `forward_iterator`, it is defined to be at the end of its underlying range if `buf_index_ + 1 == buf_last_` and `curr() == last_`. Otherwise, it is defined to be at the end of its underlying range if `buf_index_ == buf_last_` and `curr() == last_`.

If `operator*` is invoked while *utf-iterator* is at the end of its underlying range, the behavior is erroneous and the result is unspecified. Otherwise, `operator*` returns `buf_[buf_index_]`.

If `operator++` is invoked while *utf-iterator* is at the end of its underlying range, the behavior is erroneous and the iterator's state does not change. If `operator--` is invoked while *utf-iterator* is at the beginning of its underlying range, the behavior is erroneous and the iterator's state does not change.

`to_utf8_view` produces a UTF-8 view of the elements from a *utf-range*. `to_utf16_view` produces a UTF-16 view of the elements from a *utf-range*. `to_utf32_view` produces a UTF-32 view of the elements from a *utf-range*.

The names `to_utf8`, `to_utf16`, and `to_utf32` denote range adaptor objects (`[range.adaptor.object]`). `to_utf` denotes a range adaptor object template. `to_utf8` produces `to_utf8_views`, `to_utf16` produces `to_utf16_views`, and `to_utf32` produces `utf32_views`. `to_utf<ToType>` is equivalent to `to_utf8` if `ToType` is `char8_t`, to `to_utf16` if `ToType` is `char16_t`, and to `to_utf32` if `ToType` is `char32_t`. Let `to_utfN` denote any one of `to_utf8`, `to_utf16`, and `to_utf32`, and let `V` denote the `to_utfN_view` associated with that object. Let `E` be an expression and let `T` be `remove_cvref_t<decltype((E))>`. If `decltype((E))` does not model *utf-range*, `to_utfN(E)` is ill-formed. The expression `to_utfN(E)` is expression-equivalent to:

- If `T` is a specialization of `empty_view` (`[range.empty.view]`), then `empty_view<ToType>{}`.
- Otherwise, if `T` is an array type of known bound, then:
 - If the array extent is nonzero and the last element of the array is zero, then `V(std::ranges::subrange(std::ranges::begin(E), --std::ranges::end(E)))`
 - Otherwise, `V(std::ranges::subrange(std::ranges::begin(E), std::ranges::end(E)))`
- Otherwise, `V(std::views::all(E))`

`utf_view`'s implementation of the `empty()` member function is more efficient than the one provided by `view_interface`, since `view_interface`'s implementation will construct `utf_view::begin()` and `utf_view::end()` and compare them, whereas we can simply use the underlying range's `empty()`, since a `utf_view` is empty if and only if its underlying range is empty.

§ 5.10 Add code unit views and adaptors

```
namespace std::uc {

    template<class I>
    constexpr auto iterator-to-tag() { // exposition only
        if constexpr (random_access_iterator<I>) {
            return random_access_iterator_tag{};
        } else if constexpr (bidirectional_iterator<I>) {
            return bidirectional_iterator_tag{};
        } else if constexpr (forward_iterator<I>) {
            return forward_iterator_tag{};
        } else if constexpr (input_iterator<I>) {
            return input_iterator_tag{};
        }
    }
}
```

```

    }
}

template<class I>
using iterator-to-tag-t = decltype(iterator-to-tag<I>()); // exposition only

template<typename V, typename ToType>
concept convertible-to-charN-t-view = code-unit-to<ToType> && ranges::view<V> &&
    convertible_to<ranges::range_reference_t<V>, ToType>;

template<convertible-to-charN-t-view<char8_t> V>
class as_char8_t_view : public ranges::view_interface<as_char8_t_view<V>> {
    V base_ = V(); // exposition only

    template<bool Const>
    class iterator; // exposition only
    template<bool Const>
    class sentinel; // exposition only

public:
    constexpr as_char8_t_view() requires default_initializable<V> = default;
    constexpr as_char8_t_view(V base) : base_(move(base)) {}

    constexpr V& base() & { return base_; }
    constexpr const V& base() const& requires copy_constructible<V>
    {
        return base_;
    }
    constexpr V base() && { return move(base_); }

    constexpr iterator<false> begin() { return iterator<false>{ranges::begin(base_)}; }
    constexpr iterator<true> begin() const requires ranges::range<const V>
    {
        return iterator<true>{ranges::begin(base_)};
    }

    constexpr sentinel<false> end() { return sentinel<false>{ranges::end(base_)}; }
    constexpr iterator<false> end() requires ranges::common_range<V>
    {
        return iterator<false>{ranges::end(base_)};
    }
    constexpr sentinel<true> end() const requires ranges::range<const V>
    {
        return sentinel<true>{ranges::end(base_)};
    }
    constexpr iterator<true> end() const requires ranges::common_range<const V>
    {
        return iterator<true>{ranges::end(base_)};
    }

    constexpr auto size() requires ranges::sized_range<V>
    {
        return ranges::size(base_);
    }
    constexpr auto size() const requires ranges::sized_range<const V>
    {
        return ranges::size(base_);
    }
};

template<convertible-to-charN-t-view<char8_t> V>

```

```

template<bool Const>
class as_char8_t_view<V>::iterator
    : public proxy_iterator_interface<iterator-to-tag-t<ranges::iterator_t<maybe-
        const<Const, V>>>, char8_t> {
public:
    using reference_type = char8_t;

private:
    using iterator_type = ranges::iterator_t<maybe-const<Const, V>>; // exposition
        only

    friend access;

    constexpr iterator_type& base_reference() noexcept { return it_; } // exposition
        only
    constexpr iterator_type base_reference() const { return it_; } // exposition only

    iterator_type it_ = iterator_type(); // exposition only

public:
    constexpr iterator() = default;
    constexpr iterator(iterator_type it) : it_(move(it)) {}

    constexpr reference_type operator*() const { return *it_; }
};

template<convertible-to-charN-t-view<char8_t> V>
template<bool Const>
class as_char8_t_view<V>::sentinel {
    using base = maybe-const<Const, V>; // exposition only
    using sentinel_type = ranges::sentinel_t<base>; // exposition only

    sentinel_type end_ = sentinel_type(); // exposition only

public:
    constexpr sentinel() = default;
    constexpr explicit sentinel(sentinel_type end) : end_(move(end)) {}
    constexpr sentinel(sentinel_t<!Const> i) requires Const &&
        convertible_to<ranges::sentinel_t<V>, ranges::sentinel_t<base>>;

    constexpr sentinel_type base() const { return end_; }

template<bool OtherConst>
    requires sentinel_for<sentinel_type, ranges::iterator_t<maybe-const<OtherConst,
        V>>>
    friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel&
        y) {
        return x.it_ == y.end_;
    }

template<bool OtherConst>
    requires sized_sentinel_for<sentinel_type, ranges::iterator_t<maybe-
        const<OtherConst, V>>>
    friend constexpr ranges::range_difference_t<maybe-const<OtherConst, V>> operator-
        (const iterator<OtherConst>& x, const sentinel& y) {
        return x.it_ - y.end_;
    }

template<bool OtherConst>
    requires sized_sentinel_for<sentinel_type, ranges::iterator_t<maybe-
        const<OtherConst, V>>>
    friend constexpr ranges::range_difference_t<maybe-const<OtherConst, V>> operator-
        (const sentinel& y, const iterator<OtherConst>& x) {

```

```

        return y.end_ - x.it_;
    }
};

template<class R>
as_char8_t_view(R&&) -> as_char8_t_view<views::all_t<R>>;

template<convertible-to-charN-t-view<char16_t> V>
class as_char16_t_view : public ranges::view_interface<as_char16_t_view<V>> {
    V base_ = V(); // exposition only

    template<bool Const>
    class iterator; // exposition only
    template<bool Const>
    class sentinel; // exposition only

public:
    constexpr as_char16_t_view() requires default_initializable<V> = default;
    constexpr as_char16_t_view(V base) : base_(move(base)) {}

    constexpr V& base() & { return base_; }
    constexpr const V& base() const& requires copy_constructible<V>
    {
        return base_;
    }
    constexpr V base() && { return move(base_); }

    constexpr iterator<false> begin() { return iterator<false>{ranges::begin(base_)}; }
    constexpr iterator<true> begin() const requires ranges::range<const V>
    {
        return iterator<true>{ranges::begin(base_)};
    }

    constexpr sentinel<false> end() { return sentinel<false>{ranges::end(base_)}; }
    constexpr iterator<false> end() requires ranges::common_range<V>
    {
        return iterator<false>{ranges::end(base_)};
    }
    constexpr sentinel<true> end() const requires ranges::range<const V>
    {
        return sentinel<true>{ranges::end(base_)};
    }
    constexpr iterator<true> end() const requires ranges::common_range<const V>
    {
        return iterator<true>{ranges::end(base_)};
    }

    constexpr auto size() requires ranges::sized_range<V>
    {
        return ranges::size(base_);
    }
    constexpr auto size() const requires ranges::sized_range<const V>
    {
        return ranges::size(base_);
    }
};

template<convertible-to-charN-t-view<char16_t> V>
template<bool Const>
class as_char16_t_view<V>::iterator

```

```

        : public proxy_iterator_interface<iterator-to-tag-t<ranges::iterator_t<maybe-
          const<Const, V>>>, char16_t> {
public:
    using reference_type = char16_t;

private:
    using iterator_type = ranges::iterator_t<maybe-const<Const, V>>; // exposition
        only

    friend access;

    constexpr iterator_type& base_reference() noexcept { return it_; } // exposition
        only
    constexpr iterator_type base_reference() const { return it_; } // exposition only

    iterator_type it_ = iterator_type(); // exposition only

public:
    constexpr iterator() = default;
    constexpr iterator(iterator_type it) : it_(move(it)) {}

    constexpr reference_type operator*() const { return *it_; }
};

template<convertible-to-charN-t-view<char16_t> V>
template<bool Const>
class as_char16_t_view<V>::sentinel {
    using base = maybe-const<Const, V>; // exposition only
    using sentinel_type = ranges::sentinel_t<base>; // exposition only

    sentinel_type end_ = sentinel_type(); // exposition only

public:
    constexpr sentinel() = default;
    constexpr explicit sentinel(sentinel_type end) : end_(move(end)) {}
    constexpr sentinel(sentinel<!Const> i) requires Const &&
        convertible_to<ranges::sentinel_t<V>, ranges::sentinel_t<base>>;

    constexpr sentinel_type base() const { return end_; }

template<bool OtherConst>
    requires sentinel_for<sentinel_type, ranges::iterator_t<maybe-const<OtherConst,
        V>>>
    friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel&
        y) {
        return x.it_ == y.end_;
    }

template<bool OtherConst>
    requires sized_sentinel_for<sentinel_type, ranges::iterator_t<maybe-
        const<OtherConst, V>>>
    friend constexpr ranges::range_difference_t<maybe-const<OtherConst, V>> operator-
        (const iterator<OtherConst>& x, const sentinel& y) {
        return x.it_ - y.end_;
    }

template<bool OtherConst>
    requires sized_sentinel_for<sentinel_type, ranges::iterator_t<maybe-
        const<OtherConst, V>>>
    friend constexpr ranges::range_difference_t<maybe-const<OtherConst, V>> operator-
        (const sentinel& y, const iterator<OtherConst>& x) {
        return y.end_ - x.it_;
    }
}

```



```

};

template<class R>
as_char16_t_view(R&&) -> as_char16_t_view<views::all_t<R>>;

template<convertible-to-charN-t-view<char32_t> V>
class as_char32_t_view : public ranges::view_interface<as_char32_t_view<V>> {
    V base_ = V(); // exposition only

    template<bool Const>
    class iterator; // exposition only
    template<bool Const>
    class sentinel; // exposition only

public:
    constexpr as_char32_t_view() requires default_initializable<V> = default;
    constexpr as_char32_t_view(V base) : base_(move(base)) {}

    constexpr V& base() & { return base_; }
    constexpr const V& base() const& requires copy_constructible<V>
    {
        return base_;
    }
    constexpr V base() && { return move(base_); }

    constexpr iterator<false> begin() { return iterator<false>{ranges::begin(base_)}; }
    constexpr iterator<true> begin() const requires ranges::range<const V>
    {
        return iterator<true>{ranges::begin(base_)};
    }

    constexpr sentinel<false> end() { return sentinel<false>{ranges::end(base_)}; }
    constexpr iterator<false> end() requires ranges::common_range<V>
    {
        return iterator<false>{ranges::end(base_)};
    }
    constexpr sentinel<true> end() const requires ranges::range<const V>
    {
        return sentinel<true>{ranges::end(base_)};
    }
    constexpr iterator<true> end() const requires ranges::common_range<const V>
    {
        return iterator<true>{ranges::end(base_)};
    }

    constexpr auto size() requires ranges::sized_range<V>
    {
        return ranges::size(base_);
    }
    constexpr auto size() const requires ranges::sized_range<const V>
    {
        return ranges::size(base_);
    }
};

template<convertible-to-charN-t-view<char32_t> V>
template<bool Const>
class as_char32_t_view<V>::iterator
    : public proxy_iterator_interface<iterator-to-tag-t<ranges::iterator_t<maybe-
        const<Const, V>>>, char32_t> {
public:

```

```

using reference_type = char32_t;

private:
    using iterator_type = ranges::iterator_t<maybe-const<Const, V>>; // exposition
                                only

    friend access;

    constexpr iterator_type& base_reference() noexcept { return it_; } // exposition
                                only
    constexpr iterator_type base_reference() const { return it_; } // exposition only

    iterator_type it_ = iterator_type(); // exposition only

public:
    constexpr iterator() = default;
    constexpr iterator(iterator_type it) : it_(move(it)) {}

    constexpr reference_type operator*() const { return *it_; }
};

template<convertible-to-charN-t-view<char32_t> V>
template<bool Const>
class as_char32_t_view<V>::sentinel {
    using base = maybe-const<Const, V>; // exposition only
    using sentinel_type = ranges::sentinel_t<base>; // exposition only

    sentinel_type end_ = sentinel_type(); // exposition only

public:
    constexpr sentinel() = default;
    constexpr explicit sentinel(sentinel_type end) : end_(move(end)) {}
    constexpr sentinel(sentinel<!Const> i) requires Const &&
        convertible_to<ranges::sentinel_t<V>, ranges::sentinel_t<base>>;

    constexpr sentinel_type base() const { return end_; }

    template<bool OtherConst>
        requires sentinel_for<sentinel_type, ranges::iterator_t<maybe-const<OtherConst,
            V>>>
    friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel&
        y) {
        return x.it_ == y.end_;
    }

    template<bool OtherConst>
        requires sized_sentinel_for<sentinel_type, ranges::iterator_t<maybe-
            const<OtherConst, V>>>
    friend constexpr ranges::range_difference_t<maybe-const<OtherConst, V>> operator-
        (const iterator<OtherConst>& x, const sentinel& y) {
        return x.it_ - y.end_;
    }

    template<bool OtherConst>
        requires sized_sentinel_for<sentinel_type, ranges::iterator_t<maybe-
            const<OtherConst, V>>>
    friend constexpr ranges::range_difference_t<maybe-const<OtherConst, V>> operator-
        (const sentinel& y, const iterator<OtherConst>& x) {
        return y.end_ - x.it_;
    }
};

template<class R>

```

```

as_char32_t_view(R&&) -> as_char32_t_view<views::all_t<R>>;

inline constexpr unspecified as_char8_t;

inline constexpr unspecified as_char16_t;

inline constexpr unspecified as_char32_t;
}

namespace std::ranges {

    template<class V>
        inline constexpr bool enable_borrowed_range<std::uc::as_char8_t_view<V>> =
            enable_borrowed_range<V>;

    template<class V>
        inline constexpr bool enable_borrowed_range<std::uc::as_char16_t_view<V>> =
            enable_borrowed_range<V>;

    template<class V>
        inline constexpr bool enable_borrowed_range<std::uc::as_char32_t_view<V>> =
            enable_borrowed_range<V>;
}

```

char8_view produces a view of `char8_t` elements from another view. char16_view produces a view of `char16_t` elements from another view. char32_view produces a view of `char32_t` elements from another view. Let charN_view denote any one of the views char8_view, char16_view, and char32_view.

The names `as_char8_t`, `as_char16_t`, and `as_char32_t` denote range adaptor objects ([range.adaptor.object]). `as_char8_t` produces char8_views, `as_char16_t` produces char16_views, and `as_char32_t` produces char32_views. Let `as_charN_t` denote any one of `as_char8_t`, `as_char16_t`, and `as_char32_t`, and let `V` denote the charN_view associated with that object. Let `E` be an expression and let `T` be `remove_cvref_t<decltype((E))>`. Let `F` be the format enumerator associated with `as_charN_t`. If `decltype((E))` does not model `utf_pointer<T>` and if `charN_view(E)` is ill-formed, `as_charN_t(E)` is ill-formed. The expression `as_charN_t(E)` is expression-equivalent to:

- If `T` is a specialization of `empty_view` ([range.empty.view]), then `empty_view<format-to-type-t<F>>{}`.
- Otherwise, if `T` is an array type of known bound, then:
 - If the array extent is nonzero and the last element of the array is zero, then `V(std::ranges::subrange(std::ranges::begin(E), --std::ranges::end(E)))`
 - Otherwise, `V(std::ranges::subrange(std::ranges::begin(E), std::ranges::end(E)))`
- Otherwise, `V(std::views::all(E))`.

[Example 1:

```

std::vector<int> path_as_ints = {U'C', U':', U'\x00010000'};
std::filesystem::path path = path_as_ints | as_char32_t |
    std::ranges::to<std::u32string>();
auto const& native_path = path.native();
if (native_path != std::wstring{L'C', L':', L'\xD800', L'\xDC00'}) {

```

```
    return false;
}
```

— end example]

§ 5.11 Why there are three `to_utfN_views` views plus `utf_view`, and three `as_charN_t_views`

The views in `std::ranges` are constrained to accept only `std::ranges::view` template parameters. However, they accept `std::ranges::viewable_ranges` in practice, because they each have a deduction guide that looks like this:

```
template<class R>
to_utf8_view(R &&) -> to_utf8_view<views::all_t<R>>;
```

It's not possible to make this work for any view that's a template class that accepts a template parameter other than the underlying view, because of the all-or-nothing nature of deduction guides. So we need separate `to_utfN_views` and separate `as_charN_t_views` instead of having them simply be alias templates for a hypothetical generic `to_utf_view<ToType>` or `as_charN_t_view<ToType>`, respectively.

§ 5.12 Why `as_charN_t_view` is not implemented in terms of `transform_view`

Because `transform_view` *cannot* be a `borrowed_range`, whereas `as_charN_t_view` can.

[P3117R0] attempted to extend `transform_view` to be conditionally borrowed, but its authors are not pursuing it further following [concerns](#) raised by SG9 in Tokyo 2024.

A previous revision of this paper proposed for standardization a `project_view<V, F>` view that would be like `transform_view` except that the transformation function would be an NTTP, enabling `project_view` to be a `borrowed_range`. However, this was removed because the NTTP template parameter prevents us from providing a `views::all_t` deduction guide as described in the previous section.

§ 5.13 Why `utf_view` always transcodes, even in UTF-N to UTF-N cases

You might expect that if `r` in `r | to_utfN` is already in UTF-N, `r | to_utfN` might just be `r`. This is not what the `to_utfN` adaptors do, though.

The adaptors each produce a view `utfv` that stores a view of type `V`. Further, `utfv.begin()` is always a specialization of `utf-iterator`. `utfv.end()` is also a specialization of `utf-iterator` (if `common_range<V>`), or otherwise the sentinel value for `V`.

This gives `r | to_utfN` some nice, consistent properties. With the exception of `empty_view<T>{} | to_utfN`, the following are always true:

- `r | to_utfN` produces well-formed UTF. This is true even when the input was already UTF-N. Remember, the input could have been UTF-N but had ill-formed UTF in it.
- `r | to_utfN` has a consistent API. If `r | to_utfN` were sometimes `r`, and since `r` may be a reference to an array, you'd have to use `std::ranges::begin(r)` and `::end(r)` all the time.

However, you'd probably write `r.begin()` and `r.end()`, only to one day get bitten by an array-reference `r`.

§ 5.14 Add a feature test macro

Add the feature test macro `__cpp_lib_unicode_transcoding`.

§ 5.15 Relevant Polls/Minutes

§ 5.15.1 SG16 review of P2728R7 on 2023-09-13 (Telecon)

— [Minutes](#)

No polls were taken during this review.

§ 5.15.2 SG16 review of P2728R6 on 2023-08-23 (Telecon)

— [Minutes](#)

No polls were taken during this review.

§ 5.15.3 SG9 review of **D2728R4** on 2023-06-12 during Varna 2023

— [Minutes](#)

POLL: Move `null_sentinel_t` to `std::` namespace

SF	F	N	A	SA
1	3	1	0	0

Of Authors: 1

Author's Position: F

Attendance: 9 (4 abstentions)

Outcome: Consensus in favor

POLL: Remove `null_sentinel_t::base` member function from the proposal

SF	F	N	A	SA
0	4	1	0	0

Of Authors: 1

Author's Position: F

Attendance: 8 (3 abstentions)

Outcome: Consensus in favor

POLL: `utf_iterator` should be a separate type and not nested within `utf_view`

SF	F	N	A	SA
1	2	1	0	1

Attendance: 8 (3 abstentions)

of Authors: 1

Author Position: F

Outcome: Weak consensus in favor

SA: Having a separate type complexifies the API

§ 5.15.4 SG16 review of P2728R3 on 2023-05-10 (Telecon)

— [Minutes](#)

POLL: Separate `std::null_sentinel_t` from P2728 into a separate paper for SG9 and LEWG; SG16 does not need to see it again.

SF	F	N	A	SA
1	1	4	2	1

Attendance: 12 (3 abstentions)

Outcome: No consensus; author’s discretion for how to continue.

§ 5.15.5 SG16 review of P2728R0 on 2023-04-12 (Telecon)

— [Minutes](#)

POLL: SG16 would like to see a version of P2728 without eager algorithms.

SF	F	N	A	SA
4	2	0	1	0

Attendance: 10 (3 abstentions)

Outcome: Consensus in favor

POLL: UTF transcoding interfaces provided by the C++ standard library should operate on `charN_t` types, with support for other types provided by adapters, possibly with a special case for `char` and `wchar_t` when their associated literal encodings are UTF.

SF	F	N	A	SA
5	1	0	0	1

Attendance: 9 (2 abstentions)

Outcome: Strong consensus in favor

Author’s note: More commentary on this poll is provided in the section “Discussion of whether transcoding views should accept ranges of `char` and `wchar_t`”. But note here that the authors doubt the

viability of “a special case for `char` and `wchar_t` when their associated literal encodings are UTF”, since making the evaluation of a concept change based on the literal encoding seems like a flaky move; the literal encoding can change TU to TU.

§ 5.15.6 SG16 review of P2728R0 on 2023-03-22 (Telecon)

— [Minutes](#)

No polls were taken during this review.

POLL: `char32_t` should be used as the Unicode code point type within the C++ standard library implementations of Unicode algorithms.

SF	F	N	A	SA
6	0	1	0	0

Attendance: 9 (2 abstentions)

Outcome: Strong consensus in favor

§ 6 Implementation experience

The most recent revision of this paper has a reference implementation called [UtfView](#) available on GitHub, which is a fork of Jonathan Wakely’s implementation of P2728R6 as an implementation detail for `libstdc++`.

Versions of the interfaces provided by previous revisions of this paper have also been implemented, and re-implemented, several times over the last 5 years or so, as part of a proposed (but not yet accepted!) Boost library, [Boost.Text](#). Boost.Text has hundreds of stars on GitHub.

Both libraries have comprehensive tests.

§ 7 Appendix: Implementing Existing Practice for Error Handling

§ 7.1 `iconv`

This function transcodes until it finds an invalid or truncated sequence, erroring out if so and distinguishing those two cases using `errno`. It uses an out-parameter to point to the beginning of the invalid sequence.

```
struct iconv_t {};  
  
// For the sake of simplicity, this iconv only converts between UTF-8 and UTF-32.  
size_t iconv(iconv_t cd, const char** inbuf, size_t* inbytesleft, char** outbuf,  
             size_t* outbytesleft) {  
    if (!inbuf) {  
        return 0;  
    }  
    if (inbuf && !*inbuf) {  
        return 0;  
    }  
}
```

```

assert(inbytesleft);
assert(outbuf);
assert(*outbuf);
assert(outbytesleft);
    auto view = std::ranges::subrange(*inbuf, *inbuf + *inbytesleft) |
        std::uc::to_utf32;
    for (auto it = std::ranges::begin(view), end = std::ranges::end(view); it != end; ++it) {
        if (it.success()) {
            if (*outbytesleft < sizeof(char32_t)) {
                errno = E2BIG;
                return static_cast<std::size_t>(-1);
            }
            char32_t c = *it;
            (*outbuf)[0] = static_cast<char>((c >> 24) & 0xFF);
            (*outbuf)[1] = static_cast<char>((c >> 16) & 0xFF);
            (*outbuf)[2] = static_cast<char>((c >> 8) & 0xFF);
            (*outbuf)[3] = static_cast<char>(c & 0xFF);
            *outbuf += sizeof(char32_t);
            *outbytesleft -= sizeof(char32_t);
            ++it;
            std::size_t bytes_converted = it.base() - *inbuf;
            *inbytesleft -= bytes_converted;
            *inbuf = it.base();
        } else {
            transcoding_error e = it.success().error();
            switch (e) {
                case transcoding_error::truncated_utf8_sequence: {
                    errno = EINVAL;
                } break;
                case transcoding_error::unexpected_utf8_continuation_byte:
                case transcoding_error::overlong:
                case transcoding_error::encoded_surrogate:
                case transcoding_error::out_of_range:
                case transcoding_error::invalid_utf8_leading_byte: {
                    errno = EILSEQ;
                } break;
                case transcoding_error::unpaired_high_surrogate:
                case transcoding_error::unpaired_low_surrogate: {
                    std::unreachable();
                }
            }
            return static_cast<std::size_t>(-1);
        }
    }
    return 0;
}

```

§ 7.2 ICU u_strFromUTF8WithSub

This function transcodes until it finds an invalid sequence and if it does, it supports either erroring out or producing a substitution character of the user's choice. It also supports pre-flighting to determine the required output buffer size, and relying on null termination if the user doesn't supply the size of the input buffer.

```

constexpr char16_t* u_strFromUTF8WithSub(
    char16_t* dest, int32_t destCapacity, int32_t* pDestLength,
    const char* src, int32_t srcLength, char32_t subchar,
    int32_t* pNumSubstitutions, UErrorCode* pErrorCode) {
    if (*pErrorCode != U_ZERO_ERROR) {
        return nullptr;
    }
}

```



```

}
if ((src == nullptr && srcLength != 0) || srcLength < -1 || (destCapacity < 0) ||
    (dest == nullptr && destCapacity > 0) || subchar > 0x10ffff ||
    (0xD800 <= subchar && subchar <= 0xDFFF)) {
    *pErrorCode = U_ILLEGAL_ARGUMENT_ERROR;
    return nullptr;
}

if (pNumSubstitutions != nullptr) {
    *pNumSubstitutions = 0;
}

auto impl =
    [&](auto view) {
        auto end = std::ranges::end(view);
        if (pDestLength) {
            *pDestLength = 0;
            for (auto it = std::ranges::begin(view); it != end; ++it) {
                *pDestLength += it.success() ? 1 : (subchar > 0xFFFF ? 2 : 1);
            }
        }
        if (destCapacity == 0) {
            return dest;
        }
        char16_t* out_ptr = dest;
        for (auto it = std::ranges::begin(view); it != end; ++it) {
            auto write =
                [&](char16_t c) {
                    *out_ptr = c;
                    ++out_ptr;
                    --destCapacity;
                };
            if (it.success()) {
                if (destCapacity == 0) {
                    return dest;
                }
                write(*it);
            } else {
                if (subchar == -1) {
                    *pErrorCode = U_INVALID_CHAR_FOUND;
                    return dest;
                } else {
                    ++*pNumSubstitutions;
                    if (subchar > 0xFFFF) {
                        std::array<char16_t, 2> subchar_utf16{};
                        std::ranges::copy(std::array{subchar} | std::uc::to_utf16,
subchar_utf16.data());
                        write(subchar_utf16[0]);
                        if (destCapacity == 0) {
                            return dest;
                        }
                        write(subchar_utf16[1]);
                    } else {
                        write(static_cast<char16_t>(subchar));
                    }
                }
            }
        }
    }

    if (destCapacity > 0) {
        *out_ptr = char16_t{};
    }
    return dest;
}

```

```

};

if (srcLength == -1) {
    return impl(std::null_term(src) | std::uc::to_utf16);
} else {
    return impl(std::ranges::subrange(src, src + srcLength) | std::uc::to_utf16);
}
}

```

§ 7.3 Windows MultiByteToWideChar

This function transcodes until it finds an invalid sequence. If it does, it will error out if the user provides a flag; if this flag is not provided, the behavior depends on the OS. Before Windows Vista, it simply drops the invalid sequences; afterwards, it substitutes with U+FFFD. It also supports pre-flightting to determine the required output buffer size, and relying on null termination if the user doesn't supply the size of the input buffer.

```

constexpr int MultiByteToWideChar(unsigned int CodePage, unsigned long dwFlags,
                                   const char* lpMultiByteStr, int cbMultiByte,
                                   wchar_t* lpWideCharStr, int cchWideChar) {
    (void)CodePage; // For simplicity we only implement CP_UTF8
    auto impl = [&](auto view) {
        auto end = std::ranges::end(view);
        if (cchWideChar == 0) {
#ifdef WINDOWS_XP
            int chars = 0;
            for (auto it = std::ranges::begin(view); it != end; ++it) {
                chars += it.success() ? 1 : 0;
            }
            return chars;
#else
            return static_cast<int>(std::ranges::distance(view));
#endif
        } else {
            wchar_t* out_ptr = lpWideCharStr;
            for (auto it = std::ranges::begin(view); it != end; ++it) {
                auto write =
                    [&](auto c) {
                        *out_ptr = static_cast<wchar_t>(c);
                        ++out_ptr;
                        --cchWideChar;
                    };
                if (it.success()) {
                    if (cchWideChar == 0) {
                        SetLastError(ERROR_INSUFFICIENT_BUFFER);
                        return 0;
                    }
                    write(*it);
                } else {
                    if (dwFlags == MB_ERR_INVALID_CHARS) {
                        SetLastError(ERROR_NO_UNICODE_TRANSLATION);
                        return 0;
                    }
                }
            }
#ifdef WINDOWS_XP
            if (cchWideChar == 0) {
                SetLastError(ERROR_INSUFFICIENT_BUFFER);
                return 0;
            }
            write(*it);
#endif
        }
    };
}

```

```

    }
}
return static_cast<int>(out_ptr - lpWideCharStr);
}
};
if (cbMultiByte == -1) {
    if constexpr (sizeof(wchar_t) == 2) {
        return impl(std::null_term(lpMultiByteStr) | std::uc::to_utf16);
    } else {
        return impl(std::null_term(lpMultiByteStr) | std::uc::to_utf32);
    }
} else {
    if constexpr (sizeof(wchar_t) == 2) {
        return impl(std::ranges::subrange(lpMultiByteStr, lpMultiByteStr + cbMultiByte)
            |
            std::uc::to_utf16);
    } else {
        return impl(std::ranges::subrange(lpMultiByteStr, lpMultiByteStr + cbMultiByte)
            |
            std::uc::to_utf32);
    }
}
}
}
}

```

§ 7.4 Python decode()

This is a C++ analog of Python's decode function. It accepts a `std::basic_string_view`, transcodes it from UTF-8, returns a new transcoded `std::basic_string`, and throws an exception if it encounters invalid UTF which explains the problem and provides the position of the offending sequence.

```

template <typename FromChar, typename ToChar>
std::basic_string<ToChar> decode(std::basic_string_view<FromChar> input) {
    std::basic_string<ToChar> result;
    result.reserve(input.size()); // like what size_hint does
    auto view = input | to_utf<ToChar>;
    for (auto it = std::ranges::begin(view), end = std::ranges::end(view); it != end;
        ++it) {
        if (it.success()) {
            result.push_back(*it);
        } else {
            auto pos_curr = it.base() - input.begin();
            auto it2 = it;
            auto pos_next = (++it2).base() - input.begin();
            std::ostringstream ss;
            ss << "can't decode ";
            if (pos_next > pos_curr + 1) {
                ss << "characters";
            } else {
                ss << "character 0x" << std::hex
                    << static_cast<unsigned int>(static_cast<unsigned char>(*it2.base()))
                    << std::dec;
            }
            ss << " in position " << pos_curr;
            if (pos_next > pos_curr + 1) {
                ss << "-" << pos_next - 1;
            }
            ss << ": ";
            ss << [&] {
                switch (it2.success().error()) {
                    case transcoding_error::truncated_utf8_sequence:
                        return "unexpected end of data";
                }
            };
        }
    }
}

```

```

    case transcoding_error::unpaired_high_surrogate:
    case transcoding_error::unpaired_low_surrogate:
        return "illegal UTF-16 surrogate";
    case transcoding_error::unexpected_utf8_continuation_byte:
    case transcoding_error::invalid_utf8_leading_byte:
        return "invalid start byte";
    case transcoding_error::encoded_surrogate:
        if constexpr (std::same_as<FromChar, char32_t>) {
            return "code point in surrogate code point range(0xd800, 0xe000)";
        }
    case transcoding_error::overlong:
        if constexpr (std::same_as<FromChar, char32_t>) {
            return "code point not in range(0x110000)";
        }
    case transcoding_error::out_of_range:
        return "invalid continuation byte";
    }
    std::unreachable();
}();
throw std::runtime_error(std::move(ss).str());
}
}
return result;
}

```

§ 8 Special Thanks

Zach Laine, for writing revisions one through six of the paper and implementing Boost.Text.

Jonathan Wakely, for implementing P2728R6, and design guidance.

Robert Leahy, for extensive design guidance including suggesting the error handling approach introduced in R7.

Gašper Ažman, for suggesting the use of `std::expected<void, E>`.

§ 9 References

[P1629R1] JeanHeyd Meneide. 2020-03-02. Transcoding the world - Standard Text Encoding.

<https://wg21.link/p1629r1>

[P2727R4] Zach Laine. 2024-02-05. `std::iterator_interface`.

<https://wg21.link/p2727r4>

[P2871R3] Alisdair Meredith. 2023-12-18. Remove Deprecated Unicode Conversion Facets From C++26.

<https://wg21.link/p2871r3>

[P2873R2] Alisdair Meredith, Tom Honermann. 2024-07-06. Remove Deprecated locale category facets for Unicode from C++26.

<https://wg21.link/p2873r2>

[P2996R5] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2024-08-14. Reflection for C++26.

<https://wg21.link/p2996r5>

[P3117R0] Zach Laine, Barry Revzin. 2024-02-15. Extending Conditionally Borrowed.

<https://wg21.link/p3117r0>